

Containerization

A container is not a lightweight virtual machine. It is an ordinary process that Linux kernel mechanisms isolate.

Ondřej (Ondra) Krajíček

me@ondrejkrasicek.com · www.linkedin.com/in/OndrejKrajicek

v5 · 37b395f62 · built 2026-09-30

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

Two commands, one flag apart

One flag is the only difference between these two commands. Only the second one works.

DOCKER

```
docker rm -f web 2>/dev/null
docker run --rm -d --name web nginx:alpine
docker exec web sh -c \
    'apk add -q strace; strace -p 1'
# refused

docker rm -f web
docker run --rm -d --name web \
    --cap-add SYS_PTRACE nginx:alpine
docker exec web sh -c \
    'apk add -q strace; strace -p 1'
```

PODMAN

```
podman rm -f web 2>/dev/null
podman run --rm -d --name web nginx:alpine
podman exec web sh -c \
    'apk add -q strace; strace -p 1'
# refused

podman rm -f web
podman run --rm -d --name web \
    --cap-add SYS_PTRACE nginx:alpine
podman exec web sh -c \
    'apk add -q strace; strace -p 1'
```

- Without the flag, strace is refused inside the container. With the flag it works.
- Both engines answer the same way. You run both commands in Part II.

Roadmap

Seven parts. Part II runs containers. Part III shows how they work.

I · The need

Three problems, all older than the word.

II · Run your first container

Install an engine, run it, keep the data, repeat the run.

III · What is underneath

Five kernel mechanisms, five answers.

IV · Two architectures

The same mechanisms, assembled differently.

V · What follows

Why macOS and Windows need Linux inside.

VI · Where it breaks

The boundary, the escapes, the leaks.

VII · Where it came from

Optional. Timelines.

Takeaways

- 1** **No kernel object** Unrelated kernel interfaces restrict an ordinary process.
- 2** **Not a small machine** A container shares the host kernel. A virtual machine runs its own.
- 3** **Same parts, two shapes** Same mechanisms, different owner of the container process.
- 4** **Linux, or a Linux inside** A Linux container on macOS or Windows runs inside a Linux virtual machine.
- 5** **Configuration, not magic** What it may do is written in settings you can read and change.

In scope, and out of scope

COVERED

- The five kernel mechanisms, each observed with a command.
- Running one engine on your own laptop, from a first run to a repeatable setup.
- Docker against Podman, by architecture.
- Where the abstraction leaks, and the escapes recorded through it.

NOT COVERED

- Orchestration across several machines.
- Container networking beyond one published port.
- Image hardening, image size and the build cache.
- Supply-chain practice.

PART I

The need

Three problems, all older than the word container.

One machine, many workloads

One server runs several workloads, and three conditions must hold at once.

- No application reads another application's data.
- No application changes how another one runs.
- No application takes so much of the machine that the others stop.

Partial answer

An old call named chroot shows a process one subtree of the file tree and hides the rest.

Why it fails

A process inside chroot still lists the other processes and reaches the network.

Isolation for workloads that cooperate, and one defence among several. Every container uses the host kernel, so it is not a wall.

One machine, two applications with different needs

Two applications on one host need different versions of the same thing.

- The Solaris Zones paper records two applications at one customer site, both hard-coded to listen on port 80. The customer bought two servers.
- Hard-coded paths and log locations can make two versions of one application hard to install side by side.
- Boettiger cites a University of Arizona study: less than half of the software published with computer-systems papers could be built or installed.

The need is a package that carries the whole user-space environment with it.

A virtual machine solves both, and what it costs

A virtual machine is a simulated computer that runs its own kernel, so the guest kernel is not the host kernel.

11 s

to start a virtual machine, in a measurement Felter's paper cites

under 1 s

to start a container, on the same hardware

1

kernel per virtual machine, booted and held in memory

- Felter's own measurements found containers equal to or better than virtual machines on almost every workload, and both needed tuning for heavy input and output.
- Speed is not the argument. Start-up time and density are.

A third answer, between the two

A microVM is a virtual machine with almost every simulated device removed. It keeps its own kernel and gives up the rest.

	CONTAINER	MICROVM	FULL VIRTUAL MACHINE
Kernel	the host's	its own	its own
Boundary	the system-call interface	hardware, plus the monitor	hardware, plus the monitor
Start	fastest	fast	slowest

- Firecracker is one microVM monitor, and it runs a minimal guest kernel of its own.
- Its authors report a boot to application code in under 125 ms, and under 5 MB of added memory for each container.
- AWS runs Lambda and Fargate on it, and states it built Firecracker rather than choose between isolation and compatibility.

What a container is

A container is a process, or a tree of processes, that runs with its own view of the machine on the host's kernel.

Its own view

Its own files, its own process list, its own network.

Its own share

A set amount of memory and of processor time.

The host's kernel

No second kernel boots. It starts as fast as an ordinary program.

- This course's own Linux toolchain is an image. You start a container from it in Part II.

PART II

Run your first container

Install an engine, run a container, keep its data, and make the run repeatable.

The engine is the program you install

A container engine takes a command, finds the files the application needs, and starts the container.

- 1** **Resolve** Turn the name `python:3.12-slim` into a list of files to fetch from a server.
 - 2** **Unpack** Fetch the missing files and unpack them.
 - 3** **Configure** Write a configuration naming what the process may see and use.
 - 4** **Start** Start the process with that configuration.
- Docker and Podman are container engines, and both take the same verbs from the command line.

Linux: install the engine directly

Linux runs containers natively. Install one engine, and follow it for the rest of the topic.

DOCKER

```
# 1. remove the distribution's packages
sudo apt remove docker.io docker-compose
# 2. run Docker's own install script
curl -fsSL https://get.docker.com \
  -o get-docker.sh
sudo sh get-docker.sh
# 3. let your account reach the daemon
sudo usermod -aG docker $USER
newgrp docker
docker run --rm hello-world
```

PODMAN

```
sudo apt install podman
# or: sudo dnf install podman
podman --version
podman run --rm hello-world
```

If Podman refuses

Podman 6 needs version 2 of the kernel's resource-limit interface. `stat -fc %T /sys/fs/cgroup` prints `cgroup2fs` for version 2.

Version

A distribution package can be older than Podman 6. Read `podman --version`.

macOS first: Homebrew installs the command-line tools

Homebrew is a package manager for macOS. It installs command-line programs, as apt does on Ubuntu.

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

1 · Run the installer

One command, taken from brew.sh. It asks for your password once.

2 · Next steps

The installer prints lines under "Next steps". Run them. They put brew on your PATH.

3 · Where it installs

/opt/homebrew on Apple Silicon, /usr/local on an Intel Mac. The PATH lines differ for that reason.

- Check the installation with `brew --version`. Only then run `brew install lima`.

macOS: a Linux virtual machine, driven from the shell

macOS has no Linux kernel. The engine runs inside a Linux virtual machine that Lima creates.

DOCKER

```
brew install lima
limactl create template:docker
limactl start docker
limactl shell docker \
  docker run --rm hello-world
```

Template syntax

Read-only home

Intel Mac

PODMAN

```
brew install lima
limactl create template:podman
limactl start podman
limactl shell podman \
  podman run --rm hello-world
```

Current Lima takes `template:docker`, not the older `template://docker`.

Lima mounts your home directory read-only. A write there fails by design.

Podman 6 dropped Intel Macs. Use the Lima docker template there.

Windows: WSL 2, and two settings to check

Windows Subsystem for Linux runs a real Linux kernel in a small virtual machine.

DOCKER

```
wsl --install -d Ubuntu
# then, in the Ubuntu shell:
curl -fsSL https://get.docker.com \
  -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER
newgrp docker
docker run --rm hello-world
```

PODMAN

```
wsl --install -d Ubuntu
# then, in the Ubuntu shell
sudo apt install podman && \
  podman run --rm hello-world
```

Virtualisation

Error 0x80370102: turn on Virtual Machine Platform, then BIOS virtualisation.

Systemd, for Podman

Podman without root needs the Linux service manager. Set systemd=true under [boot], then run `wsl --shutdown`.

macOS and Windows needed a virtual machine

A container on macOS or Windows needs a Linux virtual machine, and that machine costs a boot only once.

WHY

The mechanisms are Linux features

- The kernel features a container needs exist in the Linux kernel.
- The macOS and Windows kernels do not have them.
- So Lima and WSL 2 supply a Linux kernel, inside a virtual machine.

WHAT IT BUYS BACK

One machine, many containers

- The virtual machine boots once.
- Each container after that costs one process start, not one boot.

One shell, and the same commands after it

Every command from here runs in a Linux shell. Open that shell once, and stay in it.

Linux

Your own terminal. Nothing to open.

macOS

`limactl shell docker`, or `limactl shell podman`, opens a shell inside the virtual machine.

Windows

`wsl` opens a shell inside the Linux distribution.

- Make your working files in that shell. A program reads a file on the macOS or Windows side more slowly than one inside the Linux filesystem, and Part VI gives the measurement.
- On Windows the subsystem supplies the ownership of files under `/mnt/c`.

An image is a stack of layers

An image is the set of files an application runs in. Those files are stored as layers.

- | | | |
|----------|-----------------|--|
| 1 | Layer | One set of file changes: files added, replaced or removed. |
| 2 | Stack | A file in a higher layer hides the file of the same name below it. |
| 3 | Manifest | The list that names each layer by a hash of its content. |
- Every layer is read-only. The files in an image never change after the build.

An image is not a container

Two words, and every later sentence depends on telling them apart.

IMAGE

Inert data

- A stack of read-only layers, plus the manifest that names them.
- Nothing about it runs. It can sit in a file or on a server.

CONTAINER

A running process

- A process, or a tree of processes, started from an image.
- It has a process identifier on the host.
- It ends when its command ends.

You cannot download a container. You download an image and start one.

A registry is where images come from

A registry stores images under a name and a hash, and serves them over a documented protocol.

DOCKER

```
docker pull python:3.12-slim
```

PODMAN

```
podman pull python:3.12-slim
```

- The engine turns the name into a manifest, then fetches the layers the manifest names.
- The name has two halves. `python` is the repository, and `3.12-slim` is the tag.
- Layers already on disk are not fetched again.
- Docker Hub allows an unauthenticated address 100 pulls in six hours, and a room behind one shared address shares that budget.
- Pull the images before you need them. Pull 101 returns an error that looks like broken Wi-Fi.

Predict: where does the container appear?

Write your answer down before anything runs.

DOCKER

```
docker rm -f web 2>/dev/null
docker run --rm -d --name web \
  -p 8080:80 nginx:alpine
curl -s localhost:8080 | head -2
```

PODMAN

```
podman rm -f web 2>/dev/null
podman run --rm -d --name web \
  -p 8080:80 nginx:alpine
curl -s localhost:8080 | head -2
```

Question 1

Does the web server appear in the host process list? Does it survive closing the terminal?

Question 2

Run ps inside the container. How many processes does it see?

- --rm deletes the container when it stops. -d returns your shell at once.
- --name labels it. -p 8080:80 maps host port 8080 to port 80 inside.

Observe: the same question, three platforms

LINUX

host shell

The web server is in the host process list.

`ps -ef` on your own machine

MACOS

Lima shell

The host list shows one virtual machine. The server is inside it.

`limactl shell` then `ps -ef`

WINDOWS

WSL shell

The same shape as macOS. Run the command inside the Linux shell.

`wsl` then `ps -ef`

- Inside the container, `ps` shows one or two processes and nothing else. On the host the same processes are there, under different numbers.

Explain: the engine started a process

The engine created nothing new. It started a process with an unusual view of the machine.

ON THE HOST

An ordinary process

- It has a process identifier.
- It can be listed, signalled and traced.
- The engine started it.

INSIDE

A very short process list

- The command is process number 1.
- Every other process on the machine is missing.
- The hostname is different too.

Open question 1: one process, two process lists. What makes the second one short?

Predict: what survives stop, start and remove?

The application writes a file. Write down what happens to that file.

DOCKER

```
docker rm -f web          # clear step 1's container
docker run -d --name web \
  nginx:alpine            # no --rm: it survives a stop
docker exec web sh -c \
  'echo hello > /data.txt'
docker stop web
docker start web
docker exec web cat /data.txt
docker rm -f web
docker run --rm nginx:alpine \
  cat /data.txt
```

PODMAN

```
podman rm -f web          # clear step 1's container
podman run -d --name web \
  nginx:alpine            # no --rm: it survives a stop
podman exec web sh -c \
  'echo hello > /data.txt'
podman stop web
podman start web
podman exec web cat /data.txt
podman rm -f web
podman run --rm nginx:alpine \
  cat /data.txt
```

Three questions

Is /data.txt there after stop and start? Is it there after remove and a new run? Did the image change?

Observe: it survives one of the two

kept

after stop and start, same container

gone

after remove, in a new container

0

changes to the image, either way

- Stopping a container does not delete it. Removing it does.
- The new container started from the same image, and the image never held the file.

The writable layer, and the image below it

An image is a stack of read-only layers. Starting a container puts one writable layer on top of that stack.

```
writable layer  ← everything the container writes, deleted with it
-----
layer 3        |
layer 2        |  read-only, shared by every container from this image
layer 1        |
```

- The topmost layer that holds the file answers a read.
- A write always lands in the writable layer.
- Remove the container and the writable layer goes with it. The image is untouched.

Open question 2: layer is the tool's word. Which kernel mechanism is it?

A bind mount shows a host directory

A bind mount makes a directory that already exists on the host appear at a path inside the container. To mount a directory is to make its contents appear at a chosen path.

```
# run this in the Linux shell from the setup. /tmp is writable on all three platforms,  
# and Lima shares your macOS home read-only, so a directory under ~ would refuse the write  
mkdir -p /tmp/work && echo hi > /tmp/work/f.txt
```

DOCKER

```
docker run --rm -v /tmp/work:/data \  
  alpine cat /data/f.txt  
docker run --rm -v /tmp/work:/data \  
  alpine touch /data/new.txt
```

PODMAN

```
podman run --rm -v /tmp/work:/data \  
  alpine cat /data/f.txt  
podman run --rm -v /tmp/work:/data \  
  alpine touch /data/new.txt
```

The pitfall: who owns the files

A bind mount can leave files under an owner nobody chose.

1 · What you see

`ls -l /tmp/work` shows `new.txt` under an unexpected owner. Under Docker's root daemon it is root.

2 · Why

Whichever account writes the file decides its owner outside. Rootless Podman maps it into the range reserved in `/etc/subuid`.

3 · Fix

Run the container under your own number with `--user $(id -u)`, or add Podman's `:U` suffix to `-v`.

4 · Windows

Under `/mnt/c`, WSL supplies the ownership.

Why a bind mount is not enough

A bind mount works. It also ties the container to one machine.

1 · One machine

The command needs this directory at this path. Another laptop does not have it.

2 · Your real files

The container can change or delete your files on the host.

3 · Across a virtual machine

On macOS and Windows the files cross into the Linux virtual machine. That is slow.

4 · Owners

Files can get an owner nobody chose.

Missing: storage the engine owns, under a name, on any machine.

A volume outlives the container

A volume is storage the engine creates and tracks under a name. Its lifetime is not the container's lifetime, and you name no host path.

DOCKER

```
docker volume create appdata
docker run --rm -v appdata:/data \
  alpine sh -c 'echo hi > /data/f'
docker run --rm -v appdata:/data \
  alpine cat /data/f
docker volume rm appdata
```

PODMAN

```
podman volume create appdata
podman run --rm -v appdata:/data \
  alpine sh -c 'echo hi > /data/f'
podman run --rm -v appdata:/data \
  alpine cat /data/f
podman volume rm appdata
```

- The first container is removed and the file is still there.
- A second container can mount the same volume and read it.
- A volume stays until you delete it.
- The same command works on any machine, because you give the volume a name and not a host path.

Observe: one write, three destinations

WRITABLE LAYER

no option

Survives stop and start.
Gone after remove.

Default for every path

VOLUME

-v
name:/path

Survives remove.
Readable by the next container.

Engine creates and tracks it

BIND MOUNT

-v
/host:/path

Survives remove.
Visible with `ls` in the shell you started in.

Host directory you name

Same write, three lifetimes. Open question 3: how does one directory appear inside one process and not the others?

The problem: a run you cannot repeat

You typed everything so far. Nothing on your machine records what you typed.

- You pulled the image by hand.
- You created the volume by hand.
- You chose the port by hand.
- A second person must repeat every command exactly, and so must you next week.
- Two files fix this. One builds the image. One records the run.

Dockerfile

A recipe that builds an image from a base image and some commands.

Compose file

A record of one run: which image, which volume, which port.

The application: a page that counts its visits

Save this as `app.py` in an empty directory. It is also in the deck's `demo/` folder.

```
# app.py: a web page that counts its visits and keeps the count in /data.
from http.server import BaseHTTPRequestHandler, HTTPServer
from pathlib import Path

COUNT = Path("/data/count.txt")

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        COUNT.parent.mkdir(parents=True, exist_ok=True)
        n = int(COUNT.read_text()) + 1 if COUNT.exists() else 1
        COUNT.write_text(str(n))
        self.send_response(200)
        self.send_header("Content-Type", "text/plain")
        self.end_headers()
        self.wfile.write(f"visit {n}\n".encode())

HTTPServer(("0.0.0.0", 8080), Handler).serve_forever()
```

- With a volume on `/data`, the count survives removing the container.

A minimal Dockerfile

Save this as Dockerfile next to app.py. Each instruction that changes the filesystem adds one layer.

```
FROM python:3.12-slim
RUN apt-get update && apt-get install -y --no-install-recommends strace \
    && rm -rf /var/lib/apt/lists/*
COPY app.py /app/app.py
CMD ["python", "/app/app.py"]
```

DOCKER

```
docker build -t pb158-app .
docker run --rm -d --name app \
    -p 8080:8080 -v appdata:/data pb158-app
curl -s localhost:8080
```

PODMAN

```
podman build -t pb158-app .
podman run --rm -d --name app \
    -p 8080:8080 -v appdata:/data pb158-app
curl -s localhost:8080
```

Observe: one row for each instruction

DOCKER

```
docker image history pb158-app
```

PODMAN

```
podman image history pb158-app
```

IMAGE	CREATED	CREATED BY	SIZE
<id>	2 minutes ago	CMD ["python" "/app/app.py"]	0B
<id>	2 minutes ago	COPY app.py /app/app.py	412B
<id>	2 minutes ago	RUN apt-get update && apt-get install ...	4.1MB
...	3 weeks ago	the rows of the python:3.12-slim base	

- The rows read bottom to top, and the base image is at the bottom.
- The CMD row is 0 bytes. It changed no file, so it added no layer.

One file brings the same setup up

```
services:
  app:
    build: .
    ports: ["8080:8080"]
    volumes: ["appdata:/data"]
volumes:
  appdata:
```

DOCKER

```
docker rm -f app 2>/dev/null
docker compose up -d
docker compose down
```

One command builds the image, creates the volume and publishes the port.

PODMAN

```
podman rm -f app 2>/dev/null
sudo apt install podman-compose
podman compose up -d
podman compose down
```

Without podman-compose the command reports no provider.

- Compose is the one place where the engines differ in substance.
- It names its volume after the project, so the count starts again.

Predict: how much memory will it report?

You start the container with a hard memory limit. Write down what the tool inside will print.

DOCKER

```
docker run --rm --memory 256m \  
  alpine sh -c 'free -h; nproc'
```

PODMAN

```
podman run --rm --memory 256m \  
  alpine sh -c 'free -h; nproc'
```

Question 1

Does `free -h` inside print 256 MB, or the whole machine? Does `nproc` count the host processors?

Question 2

Does the limit actually hold if the program allocates 512 MB?

- Keep the figure small. On Windows the virtual machine already took half of the memory.

Observe: the limit holds, the report does not

THE LIMIT

Enforced

- Allocate 512 MB under a 256 MB limit, and the kernel kills the process.
- The kernel enforces the limit. The engine does not.

THE REPORT

The host total

- `free -h` prints the whole machine's memory, and so does `/proc/meminfo`.
- `nproc` counts the host processors. The memory limit does not change that number.

Open question 4: the limit is real and the report ignores it. Two different mechanisms are at work.

Predict: two tracing questions, not one

The same tool, two targets. Write down whether each one works.

DOCKER

```
# A: the tracer starts the target
docker run --rm alpine sh -c \
    'apk add -q strace; strace -c /bin/true'

# B: attach to a running process
docker rm -f web 2>/dev/null
docker run --rm -d --name web nginx:alpine
docker exec web sh -c \
    'apk add -q strace; strace -p 1'
```

PODMAN

```
# A: the tracer starts the target
podman run --rm alpine sh -c \
    'apk add -q strace; strace -c /bin/true'

# B: attach to a running process
podman rm -f web 2>/dev/null
podman run --rm -d --name web nginx:alpine
podman exec web sh -c \
    'apk add -q strace; strace -p 1'
```

- Case A starts its own target.
- Case B attaches to a process that is already running.

Observe: one works, one refuses

CASE A	own child	It runs, and prints the calls it saw. No extra option was given.	Usually works on a current kernel
CASE B	attach	strace: attach: ptrace(PTRACE_SEIZE, 1): Operation not permitted	Refused
CASE B AGAIN	--cap-add SYS_PTRACE	Add the option to the run above, and the same attach succeeds.	One option changed the answer

```
docker rm -f web 2>/dev/null # podman: the same, with podman
docker run --rm -d --name web --cap-add SYS_PTRACE nginx:alpine
docker exec web sh -c 'apk add -q strace; strace -p 1'
```

Open question 5: one option, and the refused command was allowed to run. What did it switch on?

Observe: what the option switched on

A capability is one named piece of root's power, such as the right to trace another process. The kernel lists a process's capabilities in `/proc/self/status`.

DOCKER

```
docker run --rm alpine sh -c '  
  apk add -q libcap  
  grep Cap /proc/self/status |  
  while read n h; do  
    echo "$n $(capsh --decode=$h)"  
  done'
```

PODMAN

```
podman run --rm alpine sh -c '  
  apk add -q libcap  
  grep Cap /proc/self/status |  
  while read n h; do  
    echo "$n $(capsh --decode=$h)"  
  done'
```

- Run it twice, once with `--cap-add SYS_PTRACE` after run.
- Without the option, the `CapEff` line has no `cap_sys_ptrace`. With it, `cap_sys_ptrace` joins the list.
- The two engines start from different lists. Docker keeps `cap_net_raw`, `cap_mknod` and `cap_audit_write`, and Podman drops them.

Five questions the commands left open

- | | | |
|----------|------------------------------|--|
| 1 | Two process lists | One process, two views. What makes the second one short? |
| 2 | The vanishing file | What is a layer, and why is the image unchanged? |
| 3 | The mounted directory | How does a directory appear inside one process only? |
| 4 | The honest limit | Why does the limit hold while the report shows the host? |
| 5 | The one option | What did the one option change? |

Five questions, five kernel mechanisms, in this order.

What is underneath

Part II raised five questions that running the commands could not answer. Part III answers them with five kernel mechanisms, in the same order.

Nothing was hidden from the host

The container process was in the host list all along. What changed was the list the process itself can read.

DOCKER

```
docker rm -f web 2>/dev/null
docker run --rm -d --name web nginx:alpine
docker inspect -f '{{.State.Pid}}' web
docker exec web ps -o pid,comm
```

PODMAN

```
podman rm -f web 2>/dev/null
podman run --rm -d --name web nginx:alpine
podman inspect -f '{{.State.Pid}}' web
podman exec web ps -o pid,comm
```

- The second command prints a host process identifier. The third prints 1, and almost nothing else.
- The host numbering did not change. A second numbering was added.
- Inside that second numbering the command is process 1, and the host's processes have no number at all.
- A process with no number in your table is a process you cannot signal.

A namespace is a kernel object, not a file

A namespace is a kernel object that wraps one global resource, so the processes inside it see their own instance. The entries under `/proc` are handles to it.

```
ls -l /proc/self/ns/  
# pid → 'pid:[4026531836]'    uts → 'uts:[4026531838]'
```

1 • The link

Each entry is a symbolic link whose target names the type and a number.

2 • The handle

Opening one returns a file handle for that namespace, and keeps it alive while open.

3 • The test

Two processes are in the same namespace when the two numbers match.

Compare the numbers for a container process and for your shell.

One idea, applied eight times

Linux defines eight namespace types. Each one wraps one global resource.

TYPE	FLAG	WHAT THE PROCESS SEES ITS OWN COPY OF
PID	CLONE_NEWPID	Process identifiers
Mount	CLONE_NEWNS	Mount points
Network	CLONE_NEWNET	Network devices, addresses and ports
User	CLONE_NEWUSER	User and group identifiers
IPC	CLONE_NEWIPC	Shared memory and message queues
UTS	CLONE_NEWUTS	The hostname
Cgroup	CLONE_NEWCGROUP	Its own position in the kernel's resource-limit tree
Time	CLONE_NEWTIME	The boot and monotonic clocks

- The eight types are independent, and UTS is the kernel's old name for the hostname.

Three system calls, all traceable

No special interface creates a namespace. Three ordinary system calls do, and strace shows them.

clone(2)

Creates a process. With one or more `CLONE_NEW*` flags it creates new namespaces at the same time.

unshare(2)

Moves the calling process into a new namespace.

setns(2)

Joins an existing namespace, named by a file descriptor from `/proc`.

```
sudo unshare --pid --fork --mount-proc ps -ef    # a two-line process list
```

- `--mount-proc` gives the new namespace its own `/proc`, which is what `ps` reads. No engine ran.
- Part IV traces both engines and names the program that makes the call.

The user namespace is the load-bearing one

Only one namespace type changes which account counts as root inside it.

- The user namespace remaps user and group numbers, so root inside it is an ordinary account outside.
- Creating any other namespace type needs administrative privilege an ordinary account does not hold.
- Creating a user namespace does not, unless the Linux distribution switches that off. Ubuntu restricts it by default from release 23.10.
- That single exception is what containers without root rest on.

Union filesystem: what it sees as root

A union filesystem presents several read-only directories plus one writable directory as a single tree.

On Linux this filesystem is `overlayfs`, and it takes three directories as mount options.

lowerdir The read-only image layers, stacked.

upperdir The container's writable directory.

merged The tree the process sees as its root.

- The kernel implements this. The engine's storage driver names which filesystem does the work.
- The union filesystem serves a read from the topmost layer that holds the file. A write goes to the writable directory.

A layer is one directory in this stack.

Layering is copy-on-write, one instruction at a time

The first write to an existing file copies it up from the image layer into the writable layer.

- 1** **Share** Every container from one image reads the same layer directories.
- 2** **Protect** The kernel mounts those directories read-only.
- 3** **Copy on write** The first write to a file copies that file into the writable layer, and the write lands on the copy.
 - The copy happens on the first write to each file, not on the first write overall.
 - `fork(2)` runs this same protocol on memory pages instead of files.

The unit of sharing is a whole file

A union filesystem works at file level, not block level. The copy is never partial.

2 GB

the overlay driver copies when a process appends one byte to a 2 GB file

128

read-only layers the default overlay driver supports at once

1

copy of a file held in memory, shared by every container reading it

- A database writing small records into an image layer therefore copies each file whole before its first change.
- It is also why fifty containers from one image cost one copy of that image's files in memory.

The mount namespace, and what a volume really is

A mount namespace gives the process its own table of mount points.

DOCKER

```
docker run --rm -v appdata:/data alpine \  
  cat /proc/self/mounts | grep /data  
# an entry the host's own table does not have
```

PODMAN

```
podman run --rm -v appdata:/data alpine \  
  cat /proc/self/mounts | grep /data  
# an entry the host's own table does not have
```

1 · The kernel part

The mount namespace. Mount points added inside it are invisible outside.

2 · The engine part

Creating the directory, remembering its name, and asking for the mount.

3 · The consequence

A volume is a host directory placed into that table, so it survives the container.

Control groups: what it can consume

A control group organises processes into a hierarchy whose use of a resource can be limited and measured.

DOCKER

```
docker run --rm --memory 256m alpine \  
  cat /sys/fs/cgroup/memory.max  
# 268435456
```

PODMAN

```
podman run --rm --memory 256m alpine \  
  cat /sys/fs/cgroup/memory.max  
# 268435456
```

- Version 2 collects every resource type in one tree, and only the tree's outermost groups may hold processes.
- The resource types that matter here are memory, processor time, process count and input or output.
- A control group constrains what a process may consume. It changes nothing a process reads.

Namespaces hide, control groups constrain

Two mechanisms, two different verbs. The memory result needs both words to explain it.

NAMESPACE

Changes what is visible

- Process identifiers, mounts, hostname, network.
- Nothing about consumption.

CONTROL GROUP

Changes what is allowed

- Memory, processor time, process count.
- Nothing about visibility.

The limit held because a control group enforced it. The report showed the host because no namespace covers that file.

Nobody gave /proc/meminfo a namespace

No kernel change fixes the memory report. A userspace workaround covers those paths instead.

- /proc/meminfo and its neighbours always display numbers from the host system.
- Docker's own documentation states that free inside a container reports the host's swap.
- A separate project, LXCFS, mounts a small userspace filesystem over those paths so the numbers match the limit.
- It is a workaround in userspace, and no kernel change replaces it.

Capabilities: how much of root is left

Linux divides the privilege traditionally held by the superuser into distinct units, each switchable on its own. One such unit is a capability.

CAP_SYS_ADMIN

Mounting filesystems, setting the hostname, and creating namespaces. Close to root on its own.

CAP_SYS_PTRACE

Tracing arbitrary processes and reading their memory. Not in either engine's default set.

CAP_NET_RAW

Raw and packet sockets. In Docker's default set and not in Podman's.

- The engine drops most capabilities before it starts the container, and you can print the set.

Read the default set from the engine's own source

The default is not folklore. Both engines declare it in one file of their source code.

Podman

Eleven capabilities, from CHOWN to SYS_CHROOT, in `pkg/config/default.go`.

Docker

The same eleven plus MKNOD, NET_RAW and AUDIT_WRITE, in `daemon/pkg/oci/caps/defaults.go`.

Both

Neither list contains SYS_PTRACE.

DOCKER

```
docker run --rm alpine sh -c \  
'grep Cap /proc/self/status'
```

PODMAN

```
podman run --rm alpine sh -c \  
'grep Cap /proc/self/status'
```

Seccomp: which calls it may issue

A seccomp filter is a small program the kernel runs before each system call. It returns allow, an error, a kill, or a notification to a tracer.

- Docker's default profile refuses about 44 of more than 300 system calls.
- mount is refused, with the stated reason that the capability already gates it.
- Installing a filter needs CAP_SYS_ADMIN, or the no_new_privs setting, which stops the process gaining privilege later.
- The capability set and the filter are coupled in practice. Granting a capability also allowlists the calls that go with it.

The refusal named a mechanism

A tracer attaches in two cases only. The Yama kernel module applies both.

- | | | |
|---|---------------------|--|
| 1 | Capability | The tracer holds CAP_SYS_PTRACE. The option grants it. |
| 2 | Relationship | Or the target is a descendant of the tracer. |
| 3 | Yama | Yama applies the limit that <code>kernel.yama.pttrace_scope</code> sets. |
| 4 | The filter | A seccomp filter refuses separately, on the call itself. |

Read the refusal. `--privileged` switches every restriction off.

One file holds all five requests

One configuration file is written before a container starts. Every mechanism in this part has a field in it.

```
{ "root":    { "path": "rootfs" },
  "process": { "capabilities": { "bounding": ["CAP_CHOWN", "CAP_KILL"] } },
  "linux": {
    "namespaces": [ { "type": "pid"}, { "type": "mount"}, { "type": "ipc"},
                     { "type": "uts"}, { "type": "network"} ],
    "cgroupsPath": "/machine.slice/container.scope",
    "resources":   { "memory": { "limit": 268435456 } },
    "seccomp":     { "defaultAction": "SCMP_ACT_ERRNO", "syscalls": [] } } }
```

- root names the tree the union filesystem assembled. process.capabilities says how much of root is left.
- namespaces says what the process can see. cgroupsPath and resources say what it can consume. seccomp says which calls it may issue.
- This is an excerpt. Each mechanism from this part has its own field, and one file describes one container.

A container is a list of requests

There is no struct container in Linux, no call that creates one, and no subsystem that owns the word.

- | | |
|-----------------------------|-------------------------------------|
| 1 · Namespaces | What the process can see. |
| 2 · Control group | What it can consume. |
| 3 · Capabilities | How much of root it keeps. |
| 4 · Seccomp filter | Which system calls it may issue. |
| 5 · Union filesystem | What it sees as its root directory. |

Remove any one and something still runs. You just have less of a container.

Two architectures

The same five requests. Two different answers to one question: which process owns the container?

Engine, OCI runtime, and the work

Two programs, two jobs. The engine writes the request. The OCI runtime issues it.

THE ENGINE

- Resolves the image name.
- Unpacks the layers.
- Writes the configuration file.

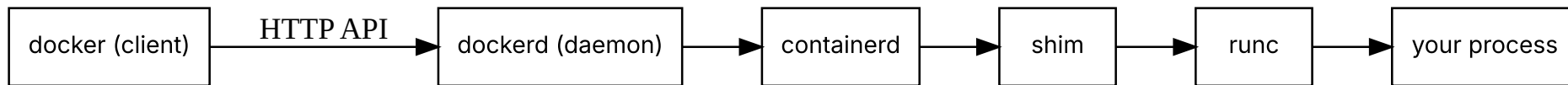
THE OCI RUNTIME

- Reads that file.
- Creates the namespaces.
- Attaches the control group.
- Sets the capability set and installs the seccomp filter.
- Mounts the volumes.
- Starts the command.

- `runc`, `crun` and `youki` are OCI runtimes. They implement one standard, published by the Open Container Initiative.
- Changing the engine does not change this tier. Both engines can use either OCI runtime.

Docker: a client, a daemon, and two tiers

A daemon is a program that keeps running after the command that started it exits, and owns resources on the user's behalf.



- The client and the daemon talk over an HTTP interface on a local socket file.
- The daemon builds, runs and distributes containers, and owns the images and volumes.
- A shim is a small process that containerd leaves in place to own one container.
- containerd was split out of the daemon to separate low-level container management.

What the host process table actually shows

The container is not a child of the daemon on a current installation. A small shim owns it.

DOCKER

```
ps -ef --forest | grep -A2 containerd-shim  
# containerd-shim-runc-v2, then the container
```

PODMAN

```
ps -ef --forest | grep -B2 nginx  
# no daemon above it, only your login session
```

Why

The daemon does not launch containers itself. A shim instance parents the container and invokes the OCI runtime.

What survives

Under Docker the container outlives your terminal, because nothing in your session owns it.

Read the process tree before trusting any drawing of it.

Stop the daemon, and its containers stop

The shim is the container's parent, but the daemon decides its lifecycle. When the daemon stops, by default it stops every container it started.

DOCKER

```
docker run -d --name web nginx:alpine
systemctl --user stop docker    # rootful: sudo systemctl stop docker
systemctl --user start docker
docker inspect -f '{{.State.Status}}' web
# exited
```

Why

One daemon owns the lifecycle of every container. If it stops or crashes, its containers stop with it.

Live restore

An optional daemon setting that keeps containers running while the daemon is down. It is off by default.

PODMAN

```
podman run -d --name web nginx:alpine
# there is no daemon to stop
podman inspect -f '{{.State.Status}}' web
# running
```

Podman: fork and exec, with a monitor left behind

Podman has no daemon. Each command is an ordinary program that sets the container up and starts the OCI runtime.

```
podman rm -f web 2>/dev/null
podman run -d --name web nginx:alpine
ps -ef --forest | grep -B1 nginx    # common, then nginx, under your session
```

- The podman process itself exits once the container is running.
- A small program named common stays behind. It started the OCI runtime, and it sits between your session and the container.
- The container is therefore a child of your session or of a systemd unit.

What a start without root actually does

Removing the daemon moves work into userspace. Podman does it in a fixed order.

- | | |
|---------------------------|--|
| 1 · Pause process | Keeps one user namespace alive for all your containers. |
| 2 · Identifier map | Reads your account's reserved user numbers and maps them. |
| 3 · Storage | Makes a mount namespace and assembles the layers. |
| 4 · Network | Starts a userspace network stack. The kernel refuses a full one. |
| 5 · OCI runtime | Execs conmon, then the OCI runtime. |

Predict: whose trace shows the namespace call?

Write your answer down. Run these in the Linux shell from the setup, as your own user.

DOCKER

```
strace -f -Y \  
-e trace=clone,clone3,unshare,setns \  
docker run --rm alpine true
```

PODMAN

```
strace -f -Y \  
-e trace=clone,clone3,unshare,setns \  
podman run --rm alpine true
```

Question 1

In which of the two traces do you see unshare?

Question 2

Which program makes the call: the one you typed, or a descendant of it?

- -Y prints the program name beside each process number. The -e trace= list keeps the trace short.
- Install the tracer inside the Linux shell first: `sudo apt install strace`, or `sudo dnf install strace`.

Observe: only one of the two traces holds the call

DOCKER

```
connect(3, ".../docker.sock") = 0  
# and no namespace call at all
```

PODMAN

```
<podman> setns(5, 0) = 0  
<runc:[1:CHILD]> unshare(  
    CLONE_NEWNS|CLONE_NEWCGROUP|  
    CLONE_NEWUTS|CLONE_NEWIPC|  
    CLONE_NEWPID) = 0
```

- Under Podman, one call creates five namespaces at once.
- The program that makes it is `runc:[1:CHILD]`, a descendant of the `podman` command you typed.
- Under Docker, the client only opens a socket to the daemon.
- The same call happens under the daemon, in a process tree your trace never entered.

Explain: the call is the same, the parent is not

DOCKER

```
sudo strace -f -Y \  
  -e trace=clone,clone3,unshare,setns \  
  -p $(pgrep -u $USER -x containerd)  
# then, in a second shell:  
docker run --rm alpine true
```

PODMAN

```
# nothing extra: the trace you  
# already ran holds the call
```

- Attached to containerd, the trace shows the same runc: [1:CHILD] calling unshare.
- strace -p joins a process that is already running.
- A Linux setting named Yama allows that only with extra privilege, unless strace started the process itself.
- The system started containerd, not your shell. So you need sudo to trace it, even when it runs under your account.

Two architectures, one system call. What differs is whose child makes it.

Two OCI runtimes, one specification

runc is written in Go. crun is written in C. They implement the same standard and are interchangeable.

3.34 s

100 sequential containers under runc,
in the crun project's own benchmark

1.69 s

the same 100 under crun, same
benchmark

512 KB

memory limit crun still ran under.
runc failed even with 4 MB

- These are the project's own figures on its own benchmark. Read them as directional.
- youki is a third OCI runtime built to the same standard.

Rootless: two different concessions

Rootless means both the engine and the container run as an unprivileged user, on top of a user namespace.

DOCKER

Keeps the daemon, moves it

- The daemon and the containers both run inside a user namespace.
- The daemon itself runs unprivileged too.
- It needs at least 65,536 reserved user numbers for your account.

PODMAN

Has no daemon to move

- Rootless is the ordinary way to run it.
- Podman states it holds no more privilege than the account that started it.
- Part VI shows a kernel flaw that no identifier mapping stops.

On Ubuntu, a security setting can stop rootless containers

Rootless containers need a user namespace. Ubuntu 24.04 gives a user namespace its full use only to programs with an AppArmor profile.

```
sysctl kernel.apparmor_restrict_unprivileged_userns # 1 means the setting is on
unshare -Ur id -u                                # a program with no profile
# unshare: write failed /proc/self/uid_map: Operation not permitted
```

- AppArmor is a Linux security module. It gives each program a profile of what the program may do.
- Ubuntu ships profiles for podman, runc, crun and rootlesskit. So the engines from the Ubuntu or vendor packages work.
- If an engine stops with the error above, it runs without a profile. Install it from the packages instead.

The setting belongs to the host. No engine option changes it.

The network path without root is a userspace stack

An unprivileged user cannot create the full network path. A program in userspace copies packets instead.

**17.9
Gbps**

one iperf3 stream, container running
as root

570 Mbps

the same stream, container without
root, in one study

- The study's authors attributed the gap to per-packet handling in `slirp4netns`.
- Podman 6 removed that stack. The shipped default is now `pasta`.
- Publishing a port without root is a userspace listener, not a firewall rule.

One image, either engine

The image format and the OCI runtime specification are written standards, not one vendor's implementation detail.

DOCKER

```
docker pull \
  docker.io/library/python:3.12-slim
```

PODMAN

```
podman pull \
  docker.io/library/python:3.12-slim
```

- The same registry, the same manifest, the same layer archives for both.
- The manifest names each layer archive by the hash of its content, and any engine can unpack them.
- A registry serves the same image to both engines over the same protocol.
- This is why a second engine can be a replacement rather than a copy.

The disagreement is about ownership

Both engines make the same five requests to the same kernel. They disagree about which process holds the container.

	DOCKER	PODMAN
Path	client to daemon to containerd to shim to runc	one command, conmon, then crun
Owner	a long-lived privileged program	your own session, or a systemd unit
Default OCI runtime	runc	crun
Without root	supported, daemon moved into a user namespace	the ordinary mode
Default capabilities	eleven, plus MKNOD, NET_RAW and AUDIT_WRITE	eleven
Buys	one interface always listening, an integrated build system	least privilege by default, no idle program
Costs	a privileged program on the path of every container	userspace shims where the kernel refuses

What follows from that

Two consequences of Part IV: the commands are portable, and macOS and Windows needed a Linux virtual machine.

Same verbs, different engine

Part II showed the commands for one engine. Almost all of them run on the other unchanged.

TASK	DOCKER	PODMAN
Run	<code>docker run</code>	<code>podman run</code>
List	<code>docker ps</code>	<code>podman ps</code>
Build	<code>docker build</code>	<code>podman build</code>
Compose	built in	needs an external provider installed
Trace	<code>--cap-add SYS_PTRACE</code>	<code>--cap-add SYS_PTRACE</code>

- The verbs match because the image format and the OCI runtime specification are standards.
- Compose is the exception. Podman ships a wrapper and no implementation.

Why macOS and Windows need a Linux virtual machine

A virtual machine runs its own kernel, so the guest kernel is not the host kernel. That is what macOS and Windows are missing.

1 · The mechanisms

All five are Linux kernel features.

2 · The gap

The macOS and Windows kernels lack them.

3 · The answer

Supply a Linux kernel in a virtual machine.

4 · Your own case

Lima and WSL 2 are instances of this, not workarounds.

Microsoft says so plainly: WSL 2 is a Linux kernel in a small virtual machine.

Where "a Linux virtual machine" stops being the answer

The claim holds for Linux containers. Windows has containers of its own, and they need a Windows kernel.

LINUX CONTAINER ON WINDOWS

Needs a Linux kernel

- Supplied by WSL 2, inside a virtual machine.
- Everything in this topic applies.

WINDOWS CONTAINER ON WINDOWS

Needs the Windows kernel

- It shares the Windows kernel directly.
Microsoft describes that as roughly how Linux containers run.
- Adding a Linux virtual machine does not help here.

One shared machine, or one for each container

How many virtual machines the engine builds is a design choice, and the answers differ.

LIMA AND WSL 2

Lima, WSL 2

One virtual machine.
Every container is a process inside it.

One kernel, many containers

APPLE

its
Containerization
project

Each Linux container runs inside its own small virtual machine.

Stronger separation, more machines

- On the second shape two containers share no kernel, so the shared-kernel claim describes the usual shape and not a law.

Where it breaks

The boundary, the escapes through it, and the leaks you can measure on your own laptop.

The boundary is the system-call interface

A virtual machine gives a guest its own kernel behind a hardware boundary. A container shares the host kernel, so its boundary is that kernel's system-call interface.

1

kernel shared by every container on the host

300+

system calls the boundary is drawn on

~44

of them refused by Docker's default profile

- Linux publishes a stable, versioned, documented system-call interface.
- Stability makes it dependable. Width makes it large to defend.
- A kernel bug is therefore a possible container escape.

A container escape, in four steps

CVE-2019-5736 overwrote the host OCI runtime binary from inside a container.

- 1 Bait** A malicious image points its entry at `/proc/self/exe`.
- 2 Re-entry** The OCI runtime re-executes itself inside it.
- 3 Handle** A library the attacker loaded opens the host's OCI runtime binary for writing.
- 4 Overwrite** The next start runs attacker code as root.
 - The attack needed only the default options.
 - It reached Docker, Podman, CRI-O and containerd alike.

One class, disclosed four times

One class of bug returned four times in runc. The container controls a path, and the OCI runtime follows it.

Nov 2025

Three flaws of one class. Each turned on a path the container controlled.

One mechanism

A link replaces `/dev/null`, so the OCI runtime mounts a host path read-write.

Another

The kernel runs a helper on the host, and that helper ignores namespaces.

2026

CVE-2026-41579. Again a `/dev` link, again limited host writes.

- An untrusted image or build file is the most likely way in. The runc 1.1.x branch stayed unpatched.

When the kernel is the bug

Dirty Pipe was a kernel flaw. No fix in an OCI runtime could have prevented it.

- CVE-2022-0847 let an unprivileged process write to any file it could read.
- A public proof of concept escaped a container that was granted no extra privilege. It overwrote the cached copy of the OCI runtime binary.
- Patching the engine would not have helped. Changing the engine would not have helped.
- This is what "the boundary is the kernel" means in practice.

Every process in every container on that host was behind the same one bug.

One systematic study, and what it counted

One published survey counted the flaws across eleven implementations instead of estimating them.

59

published flaws across the 11
"container runtimes" one survey
examined

46%

of the 28 that had a working proof of
concept enabled an escape

7 of 9

tested escapes prevented by user
namespaces

- The survey's authors put most cases in one category: a host component leaked into the container.
- A user namespace prevented seven of the nine exploits the survey tested. Two still worked.
- The two that survived involved the host OCI runtime still running as root against objects the container controls.

The other boundary is narrower, not absolute

A virtual machine's interface is smaller than a system-call interface. Smaller is not zero.

- CVE-2025-22225 is a published flaw in VMware ESXi. An attacker with enough privilege could reach the host through the process that runs the virtual machine.
- The comparison is about the width of the interface, not about a guarantee on either side.

Container

The whole system-call interface of a shared kernel.

Virtual machine

A much smaller device and instruction interface. An attacker crossed it too.

Containers went back inside virtual machines

AWS replaced Linux containers with microVMs to isolate the functions its on-demand service runs.

- AWS first isolated its on-demand function service with Linux containers, and reported itself unsatisfied.
- Their stated reason: reliance on a single kernel forces a trade-off between security and compatibility.
- They built Firecracker instead of choosing between the two.
- Kubernetes says sandboxing is often recommended for untrusted code, where workloads are assumed to be malicious.

Packaging is where containers replaced the alternative. Untrusted tenants are where they did not.

Which containers are not virtual machines

The shared-kernel claim covers the usual shape. Two documented cases fall outside it.

KATA CONTAINERS

project page	An OCI runtime that builds a small virtual machine for each container.	An engine runs it instead of runc
--------------	--	-----------------------------------

THE STANDARD

the OCI runtime specification	The runtime specification carries a virtual-machine section of its own.	Not a deviation
-------------------------------	---	-----------------

- So the claim is scoped. A namespace-based Linux container shares the host kernel, and an OCI runtime that builds a virtual machine still conforms to the same standard.

The leak you can measure: files across the boundary

On macOS a bind mount crosses out of the virtual machine on every access. A volume does not.

3.5x

slower than Linux with VirtioFS, many small files through a bind mount, in one measurement

10x

slower on the same files with gRPC-FUSE

~1x

in line with Linux, inside a named volume

- One author measured these ratios on one codebase. They show direction only.
- Many small files dominate the cost, not total bytes.
- On Windows, keep working files in the Linux filesystem.
- Two machines on one engine version differ if one uses the slower file-sharing driver.

The leak you can trip over: the wrong architecture

An image built for one processor architecture on a machine with another runs under translation, or not at all.

Clear failure

`exec format error`. The image was built for another architecture and nothing translates it here.

Unclear failure

`Illegal instruction`. The translator does not implement an instruction the program uses.

Slow success

It runs. Each system call is caught, converted and forwarded, so system-call-heavy work suffers most.

Fix

Name your own architecture with `--platform linux/arm64` or `--platform linux/amd64`. Prefer images built for that architecture.

What is inside an image nobody opened

An image freezes a package set on the day it was built, and nothing refreshes it afterwards.

- One scanning study of images built for neuroscience data analysis found they contain hundreds of known vulnerabilities.
- Rebuilding the same images against current packages removed about two thirds of those vulnerabilities.
- The same paper reports, from earlier work rather than its own, that over 30 percent of official images on one registry carry high-priority vulnerabilities.
- The images pulled in Part II are somebody else's filesystem, running under your engine.
- Rebuild on a schedule, because nothing else refreshes the package set.

Three things the evidence leaves open

- **A namespace can outlive its processes**

namespaces(7)

Bind-mounting an entry in `/proc` keeps a namespace alive after every process in it exits.

- **The memory report has no kernel fix**

Docker docs

Nothing namespaced those files. The workaround is a userspace filesystem.

- **Your own installation has its own defaults**

containers.conf

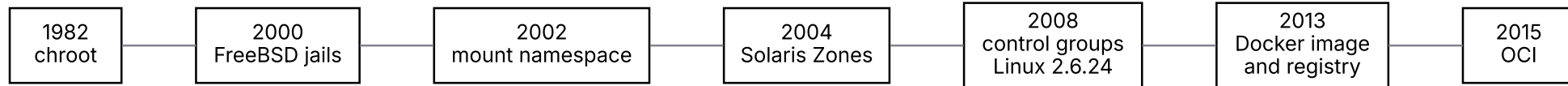
Read your network backend and storage driver from `containers.conf`.

Read the real limit from `/sys/fs/cgroup/memory.max`, not `free`. Check `/proc` for a bind mount before trusting that a namespace is gone.

Where it came from

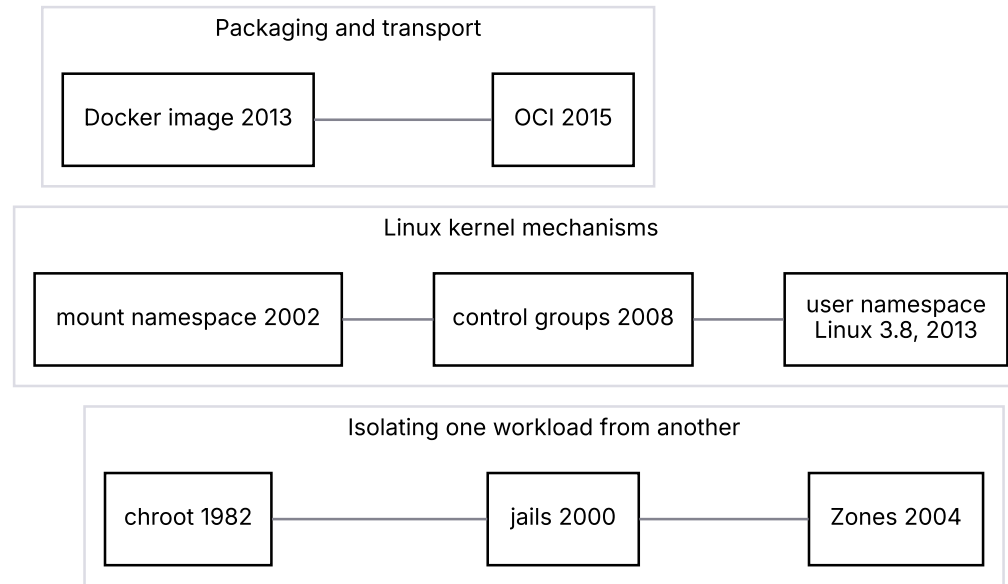
Four pictures. Optional.

One line, 1982 to 2015



- chroot shows a process one subtree of the file tree and hides the rest.
- FreeBSD jails, in 2000, answered the same need with a compartment that reached past the file tree.
- Solaris Zones, in 2004, answered the same need on Solaris, and limited what each compartment consumed.
- The chroot date comes from a reading of the source-control logs, not from folklore.
- Popular timelines print 1979. The two accounts genuinely disagree, and the log date is better evidenced.
- Control groups are dated by the kernel release that carried them, not by a press date.

Three lanes that met



- The lanes are parallel answers to one pressure, not a line of descent. Accounts written at the time attribute the idea of namespacing to Plan 9, not to BSD.
- The kernel maintainers refused the word. Google's "process containers" became control groups.

What 2013 added, and what it did not

Isolation was available for a decade before the first Docker release. What was missing was a way to move an application between machines.

NOT NEW IN 2013

The isolation

- Namespaces from 2002, control groups from 2008.
- LXC had already combined them, and Docker began as a command-line front end to LXC.

NEW IN 2013

The image and the transport

- A layered image named by the hash of its content.
- A protocol for moving it between machines.
- Deployment becomes one command against a named hash.

Kubernetes dropped Docker, and the images kept working

Since 2015 the image format and the runtime contract are standards. One consequence arrived seven years later.

- Kubernetes removed dockershim, its adapter for Docker, in release 1.24.
- Images built by Docker kept working everywhere, because they are standard OCI images.
- Docker was replaced underneath the format. The format did not move.

Docker no longer controls the implementation, and the images it builds still run under every conforming engine.

Takeaways

- 1** **No kernel object** Five mechanisms in one configuration file. Remove one and something still runs.
- 2** **Not a small machine** The boundary is a shared kernel. The record has real escapes through it.
- 3** **Same parts, two shapes** They differ over who owns the container, not over what is asked of the kernel.
- 4** **Linux, or a Linux inside** Lima and WSL 2 supply that kernel. Windows containers are the exception.
- 5** **Configuration, not magic** Read the capability set before reaching for an option that removes it.

What follows

Four things to do differently once a container stops being a black box.

- | | |
|---|---|
| 1 · Read the refusal | Find which capability or filter rule refused. Then grant that one. |
| 2 · Choose where data lives | Writable layer, volume or bind mount. Decide before the first run. |
| 3 · Check the host, not the flag | Control group version, namespace policy and architecture set the limits.
No option overrides them. |
| 4 · Keep the habit | Predict, observe, explain. |

Three of these read a value the machine already holds: the capability set, the storage destination, and the control group version.

References · 1/18

- **[S1]** namespaces(7) — Linux manual page — <https://man7.org/linux/man-pages/man7/namespaces.7.html>
- **[S2]** cgroups(7) — Linux manual page — <https://man7.org/linux/man-pages/man7/cgroups.7.html>
- **[S3]** capabilities(7) — Linux manual page — <https://man7.org/linux/man-pages/man7/capabilities.7.html>
- **[S4]** seccomp(2) — Linux manual page — <https://man7.org/linux/man-pages/man2/seccomp.2.html>
- **[S5]** Seccomp security profiles for Docker — <https://docs.docker.com/engine/security/seccomp/>
- **[S6]** Use the OverlayFS storage driver — <https://docs.docker.com/engine/storage/drivers/overlayfs-driver/>

References · 2/18

- **[S7]** OCI Runtime Specification — Linux Container Configuration — <https://github.com/opencontainers/runtime-spec/blob/main/config-linux.md>
- **[S8]** OCI Image Format Specification — <https://github.com/opencontainers/image-spec/blob/main/spec.md>
- **[S9]** About the Open Container Initiative — <https://opencontainers.org/about/overview/>
- **[S10]** Docker overview — <https://docs.docker.com/get-started/docker-overview/>
- **[S11]** Rootless mode — <https://docs.docker.com/engine/security/rootless/>
- **[S12]** Live restore — <https://docs.docker.com/engine/daemon/live-restore/>
- **[S13]** podman(1) — Podman manual page — <https://docs.podman.io/en/latest/markdown/podman.1.html>

References · 3/18

- **[S14]** podman — project README — <https://github.com/podman-container-tools/podman>
- **[S15]** What happens behind the scenes of a rootless Podman container? — <https://www.redhat.com/en/blog/behind-scenes-podman>
- **[S16]** containers/crun — <https://github.com/containers/crun>
- **[S17]** Why strace doesn't work in Docker — <https://jvns.ca/blog/2020/04/29/why-strace-doesnt-work-in-docker/>
- **[S18]** What even is a container — <https://jvns.ca/blog/2016/10/10/what-even-is-a-container/>
- **[S19]** Jails: Confining the omnipotent root — <http://phk.freebsd.dk/pubs/sane2000-jail.pdf>
- **[S20]** Jails – High value but shitty Virtualization — <http://phk.freebsd.dk/sagas/jails/>

References · 4/18

- **[S21]** Namespaces — <https://k1024.org/papers/namespaces.html>
- **[S22]** From dotCloud to Docker — <https://jpetazzo.github.io/2017/02/24/from-dotcloud-to-docker/>
- **[S23]** Docker build cache — <https://docs.docker.com/build/cache/>
- **[S24]** A container-confinement breakout — <https://lwn.net/Articles/781013/>
- **[S25]** Anatomy of CVE-2019-5736: A runc container escape — <https://aws.amazon.com/blogs/compute/anatomy-of-cve-2019-5736-a-runc-container-escape/>
- **[S26]** runc security advisory GHSA-9493-h29p-rfm2 — <https://github.com/opencontainers/runc/security/advisories/GHSA-9493-h29p-rfm2>
- **[S27]** runc container breakout vulnerabilities: a technical overview — <https://www.cncf.io/blog/2025/11/28/runc-container-breakout-vulnerabilities-a-technical-overview/>

References · 5/18

- **[S28]** runc releases — <https://github.com/opencontainers/runc/releases>
- **[S29]** Towards Improving Container Security by Preventing Runtime Escapes — <https://doi.org/10.2172/1890908>
- **[S30]** Escaping containers using the Dirty Pipe vulnerability — <https://securitylabs.datadoghq.com/articles/dirty-pipe-container-escape-poc/>
- **[S31]** An analysis of security vulnerabilities in container images for scientific data analysis — <https://academic.oup.com/gigascience/article/10/6/giab025/6291571>
- **[S32]** Containers do not contain — <https://opensource.com/business/14/7/docker-security-selinux>
- **[S33]** Kata Containers — <https://katacontainers.io/>

References · 6/18

- **[S34]** OCI Runtime Specification — Virtual Machine Container Configuration — <https://github.com/opencontainers/runtime-spec/blob/main/config-vm.md>
- **[S35]** Isolation modes for Windows containers — <https://learn.microsoft.com/en-us/virtualization/windowscontainers/manage-containers/hyperv-container>
- **[S36]** Zero-Day Vulnerabilities in VMware ESXi — <https://www.tenable.com/blog/cve-2025-22224-cve-2025-22225-cve-2025-22226-zero-day-vulnerabilities-in-vmware-esxi>
- **[S37]** Containerization — <https://github.com/apple/containerization>
- **[S38]** Firecracker: Lightweight Virtualization for Serverless Applications — <https://www.usenix.org/system/files/nsdi20-paper-agache.pdf>
- **[S39]** Dockershim Removal FAQ — <https://kubernetes.io/blog/2022/02/17/dockershim-faq/>

References • 7/18

- **[S40]** Don't Panic: Kubernetes and Docker — <https://kubernetes.io/blog/2020/12/02/dont-panic-kubernetes-and-docker/>
- **[S41]** Memory inside Linux containers — <https://fabiokung.com/2014/03/13/memory-inside-linux-containers/>
- **[S42]** containers/common — pkg/config/default.go — <https://github.com/kubernetes/kubernetes/blob/main/pkg/config/default.go>
- **[S43]** moby/moby — daemon/pkg/oci/caps/defaults.go — <https://github.com/moby/moby/blob/master/daemon/pkg/oci/caps/defaults.go>
- **[S44]** Docker on macOS is slow and how to fix it — <https://www.cncf.io/blog/2023/02/02/docker-on-macos-is-slow-and-how-to-fix-it/>

References · 8/18

- **[S45]** Rosetta emulation on arm64 macOS breaks many amd64 images — <https://github.com/docker/for-mac/issues/7137>
- **[S46]** Comparing WSL Versions — <https://learn.microsoft.com/en-us/windows/wsl/compare-versions>
- **[S47]** Systemd support in WSL — <https://learn.microsoft.com/en-us/windows/wsl/systemd>
- **[S48]** Ubuntu 24.04 LTS release notes — <https://documentation.ubuntu.com/release-notes/24.04/>
- **[S49]** Docker Hub usage and rate limits — <https://docs.docker.com/docker-hub/usage/>
- **[S50]** Docker Desktop license agreement — <https://docs.docker.com/subscription/desktop-license/>
- **[S51]** Install Docker Engine — <https://docs.docker.com/engine/install/>
- **[S52]** Podman Installation Instructions — <https://podman.io/docs/installation>

References • 9/18

- **[S53]** Lima templates — <https://lima-vm.io/docs/templates/>
- **[S54]** Lima configuration — <https://lima-vm.io/docs/config/>
- **[S55]** podman-machine-init(1) — <https://docs.podman.io/en/latest/markdown/podman-machine-init.1.html>
- **[S56]** containers-from-scratch — <https://github.com/lizrice/containers-from-scratch>
- **[S57]** Predict-Observe-Explain — <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>
- **[S58]** Solaris Zones: Operating System Support for Consolidating Commercial Workloads — https://www.usenix.org/legacy/event/lisa04/tech/full_papers/price/price.pdf

References • 10/18

- **[S59]** Moving ARM64 Docker Builds Off QEMU — <https://www.warpbuild.com/guides/qemu-vs-native-arm64-builds>
- **[S60]** OCI Image Format Specification — Image Configuration — <https://github.com/opencontainers/image-spec/blob/main/config.md>
- **[S61]** podman-run(1) — Podman command reference — <https://docs.podman.io/en/latest/markdown/podman-run.1.html>
- **[S62]** An introduction to Docker for reproducible research — https://polaris.imag.fr/arnaud.legrand/research/readings/acm_sigops_si_rsea/p71-boettiger.pdf
- **[S63]** An Updated Performance Comparison of Virtual Machines and Linux Containers — <https://www.read.seas.harvard.edu/~kohler/class/cs260r-s19/containerperf14.pdf>

References • 11/18

- **[S64]** Storage overview — Docker Docs — <https://docs.docker.com/engine/storage/>
- **[S65]** Podman v6.0.0 release notes — <https://github.com/podman-container-tools/podman/releases/tag/v6.0.0>
- **[S66]** podman-compose(1) — Podman manual page — <https://docs.podman.io/en/latest/markdown/podman-compose.1.html>
- **[S67]** ptrace(2) — Linux manual page — <https://man7.org/linux/man-pages/man2/ptrace.2.html>
- **[S68]** Resource constraints — Docker Docs — https://docs.docker.com/engine/containers/resource_constraints/
- **[S69]** lxc/lxcfs — project README — <https://github.com/lxc/lxcfs>
- **[S70]** containerd Runtime v2 (shim API) — <https://containerd.io/docs/2.3/runtime-v2/>

References • 12/18

- **[S71]** Advanced settings configuration in WSL — <https://learn.microsoft.com/en-us/windows/wsl/wsl-config>
- **[S72]** Working across Windows and Linux file systems (WSL) — <https://learn.microsoft.com/en-us/windows/wsl/filesystems>
- **[S73]** Multi-tenancy — Kubernetes documentation — <https://kubernetes.io/docs/concepts/security/multi-tenancy/>
- **[S74]** Overlay Filesystem — the Linux Kernel documentation — <https://docs.kernel.org/filesystems/overlayfs.html>
- **[S75]** What is a container? — Docker Docs — <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>

References · 13/18

- **[S76]** Volumes — Docker Docs — <https://docs.docker.com/engine/storage/volumes/>
- **[S77]** Ubuntu 23.10 restricted unprivileged user namespaces — <https://ubuntu.com/blog/ubuntu-23-10-restricted-unprivileged-user-namespaces>
- **[S78]** containers.conf(5) — configuration reference — <https://github.com/containers/common/blob/main/docs/containers.conf.5.md>
- **[S79]** Troubleshooting Windows Subsystem for Linux — <https://learn.microsoft.com/en-us/windows/wsl/troubleshooting>
- **[S80]** Bind mounts — Docker Docs — <https://docs.docker.com/engine/storage/bind-mounts/>
- **[S81]** OCI Runtime Specification — Configuration — <https://github.com/opencontainers/runtime-spec/blob/main/config.md>

References • 14/18

- **[S82]** Install Docker Engine on Ubuntu — <https://docs.docker.com/engine/install/ubuntu/>
- **[S83]** Homebrew: The Package Manager for Everywhere — <https://brew.sh>
- **[S84]** Yama — the Linux Kernel documentation — <https://docs.kernel.org/admin-guide/LSM/Yama.html>
- **[S85]** Linux post-installation steps for Docker Engine — <https://docs.docker.com/engine/install/linux-postinstall/>
- **[S86]** Homebrew — Installation — <https://docs.brew.sh/Installation>
- **[V1]** Containers with Docker and Podman — note in the author's research vault.
- **[V2]** Containers with Docker and Podman — note in the author's research vault.
- **[V3]** Containers with Docker and Podman — note in the author's research vault.

References • 15/18

- **[V4]** Containers with Docker and Podman — note in the author's research vault.
- **[V5]** Containers with Docker and Podman — note in the author's research vault.
- **[V6]** Containers with Docker and Podman — note in the author's research vault.
- **[V7]** Containers with Docker and Podman — note in the author's research vault.
- **[V8]** Containers with Docker and Podman — note in the author's research vault.
- **[V9]** Containers with Docker and Podman — note in the author's research vault.
- **[V10]** Containers with Docker and Podman — note in the author's research vault.
- **[V11]** Containers with Docker and Podman — note in the author's research vault.
- **[V12]** Containers with Docker and Podman — note in the author's research vault.

References • 16/18

- **[V13]** Containers with Docker and Podman — note in the author's research vault.
- **[V14]** Containers with Docker and Podman — note in the author's research vault.
- **[V15]** Containers with Docker and Podman — note in the author's research vault.
- **[V16]** Containers with Docker and Podman — note in the author's research vault.
- **[V17]** Containers with Docker and Podman — note in the author's research vault.
- **[V18]** Containers with Docker and Podman — note in the author's research vault.
- **[V19]** Copy-on-Write — note in the author's research vault.
- **[V20]** Copy-on-Write — note in the author's research vault.
- **[V21]** The System Call Boundary — note in the author's research vault.

References • 17/18

- **[V22]** Where the Abstraction Leaks — note in the author's research vault.
- **[V23]** Where the Abstraction Leaks — note in the author's research vault.
- **[V24]** devcontainers — note in the author's research vault.
- **[V26]** Processes, Threads and Scheduling — note in the author's research vault.
- **[V27]** Containers with Docker and Podman — note in the author's research vault.
- **[V28]** Containers with Docker and Podman — note in the author's research vault.
- **[V29]** Containers with Docker and Podman — note in the author's research vault.
- **[V30]** Containers with Docker and Podman — note in the author's research vault.
- **[V31]** Containers with Docker and Podman — note in the author's research vault.

References · 18/18

- **[V32]** Containers with Docker and Podman — note in the author's research vault.
- **[V33]** Containers with Docker and Podman — note in the author's research vault.
- **[V34]** Containers with Docker and Podman — note in the author's research vault.
- **[V35]** Containers with Docker and Podman — note in the author's research vault.
- **[V36]** Containers with Docker and Podman — note in the author's research vault.
- **[V37]** Containers with Docker and Podman — note in the author's research vault.
- **[V38]** Containers with Docker and Podman — note in the author's research vault.

The references document (PDF and web page) carries the full source list, and a summary of every [V] source with its own external references.

TAKE IT WITH YOU

Everything from this talk

[Slides \(PDF\)](#)

[Handout \(PDF\)](#)

[References \(PDF\)](#)

[References \(web page\)](#)

[Speaker notes \(PDF\)](#)

[Everything \(.zip\) — readable by an AI agent](#)

ondrasek.github.io/talks

me@ondrejkrasicek.com